



11

Debugging WinUI Applications with Visual Studio

Good debugging skills are essential for developers. While .NET developers need to know how to use features including breakpoints and the **Output** and **Immediate** windows, WinUI project debugging adds another set of tools and techniques to master. There are issues that can arise in the UI layer with data binding, layout, and resources. You will learn how to use **Live Visual Tree** and **Live Property Explorer** and how to discover data binding errors with Visual Studio's **XAML Binding Failures** window.

In this chapter, we will cover the following topics:

- How to debug WinUI applications and work with breakpoints in ViewModels and service classes
- How to debug data binding failures by leveraging the **XAML Binding Failures** window in Visual Studio and avoid common problems when binding to collections
- Explore the **Live Visual Tree** window in Visual Studio to find layout problems in your XAML
- Discover and use **Live Property Explorer** to get and set data in your XAML elements at runtime

By the end of this chapter, you will be comfortable with debugging common problems typically encountered by WinUI developers while

developing applications. You'll be able to use these skills while building applications with other XAML frameworks, such as .NET MAUI, WPF, and UWP, too.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1809 (build 17763) or newer or Windows 11
- Visual Studio 2022 or later with the .NET Desktop Development workload configured for Windows App SDK development

The source code for this chapter is available on GitHub at this URL:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter11>.

Debugging in Visual Studio

There are several fundamental areas of debugging WinUI applications that we will be covering in this section. Some of these techniques are applicable to debugging other types of .NET applications, whereas others are specific to XAML and WinUI development. Throughout the book, we've run our projects with Visual Studio, which is an example of a local debugging session. We will explore other ways to debug local applications as well as remote applications.

Simple mistakes in XAML markup are not always apparent when we're writing it, and this kind of problem is not detected by the compiler. In this section, we will see how to detect and avoid XAML markup issues and how to adhere to best practices.

Let's get started by taking a closer look at debugging local applications.

Debugging local applications

Open the **Ch11-MyMediaCollection** project from GitHub and compile it to make sure you have downloaded all the referenced NuGet packages. This is the WinUI 3 project that we've been building throughout the previous chapters. Run the application to make sure everything works as expected.

NOTE

*If you have experience in developing and debugging WPF or UWP applications, you're probably familiar with **XAML Designer** in Visual Studio. There is no designer support for WinUI 3 applications at this time. It's unclear if or when this feature will be added. There is some discussion on GitHub about this topic here:*

<https://github.com/microsoft/microsoft-ui-xaml/issues/5917>. As one of the comments in the issue suggests, you can try to work around this limitation by leveraging **XAML Hot Reload** in Visual Studio to make changes to the XAML file while debugging. While there are some limitations to this approach (see <https://github.com/microsoft/microsoft-ui-xaml/issues/7043>), most simple XAML changes will be reflected in the running app when saving. For more information about XAML Hot Reload, see the Microsoft Learn documentation: <https://learn.microsoft.com/visualstudio/xaml-tools/xaml-hot-reload>.

For our first walk-through, let's look at how to debug a local application installed on your PC.

Debugging a locally installed application

We have been running and debugging our Visual Studio solutions throughout this book. Now we will see how you can debug an application that you have already installed in Windows.

You have previously run several projects while reading this book. Unless you uninstalled them, each should appear as an installed app package that you can debug. Let's start with the steps:

1. Start by selecting **Debug | Other Debug Targets | Debug Installed App Package** in Visual Studio. The **Debug Installed App Package** window will appear:

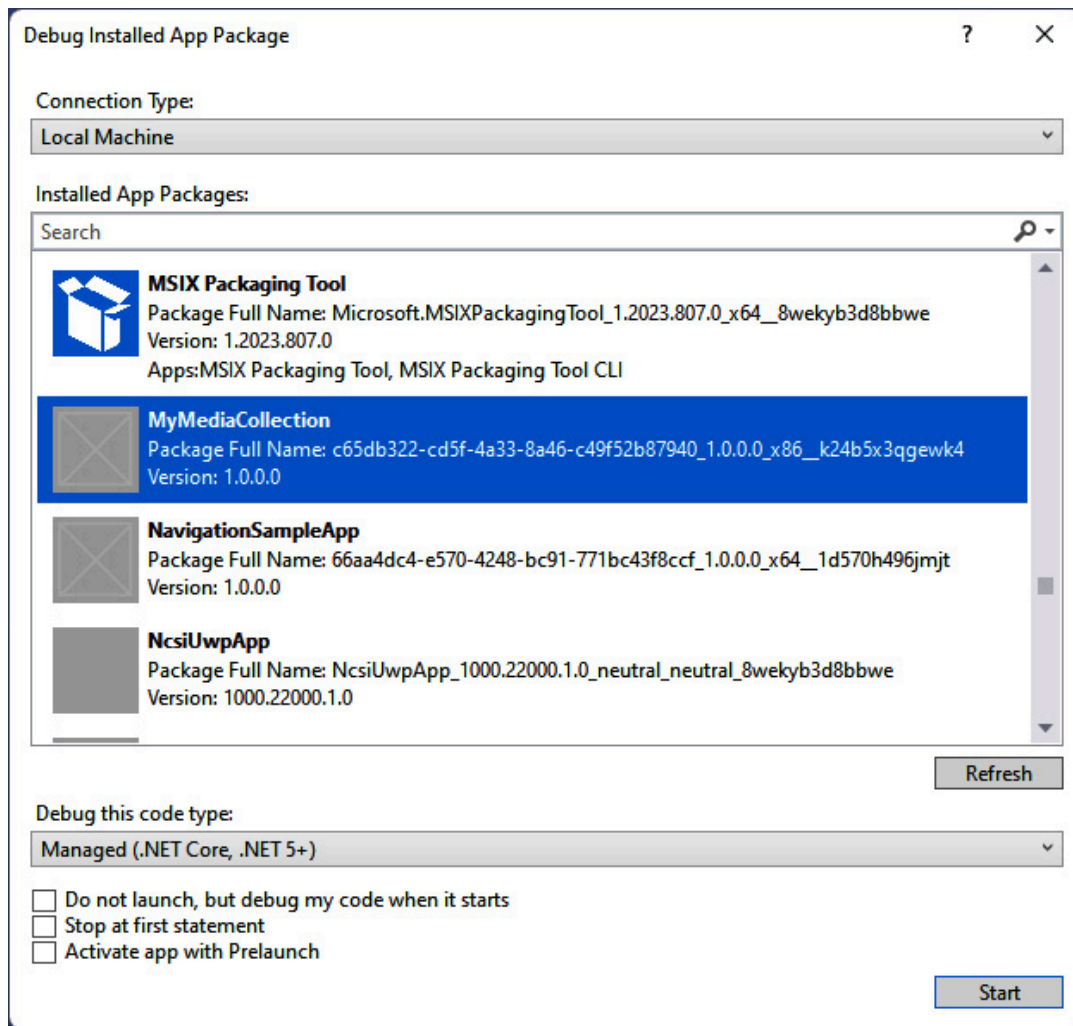


Figure 11.1 – The Debug Installed App Package window

This window will show all the installed packages on your Windows PC. Some names will be familiar, such as **Microsoft.WindowsTerminal**, **Fluent XAML Theme Editor**, or **Microsoft Defender**. Others will only be listed by their application ID. You can select some of these other applications to debug, but without debug symbols, you won't be able to

hit any breakpoints or step through the code. Let's find one of our applications.

2. If you have debugged the **MyMediaCollection** app on your machine, it will appear in the list. So, let's search for **MyMediaCollection**. The **MyMediaCollection** application will appear in the search results. If you do not see it, make sure you have run the application from Visual Studio at least once. This step is necessary to package and deploy the application to Windows.
3. Select it and click **Start**. The application will run, and Visual Studio will start debugging.

If you don't want to start debugging immediately, you can select the **Do not launch, but debug my code when it starts** checkbox. Now, Visual Studio will start debugging when you start the application from the **Start** menu or any other method.

Another way to start debugging an installed local application is by attaching it to a running application:

1. First, run the app from the **Start** menu, and in Visual Studio, go to **Debug | Attach to Process**.
2. In the **Attach to Process** window, find the **ApplicationFrameHost.exe** process with the title that matches the application you want to debug. This is the process that hosts every packaged WinUI and UWP application on Windows:

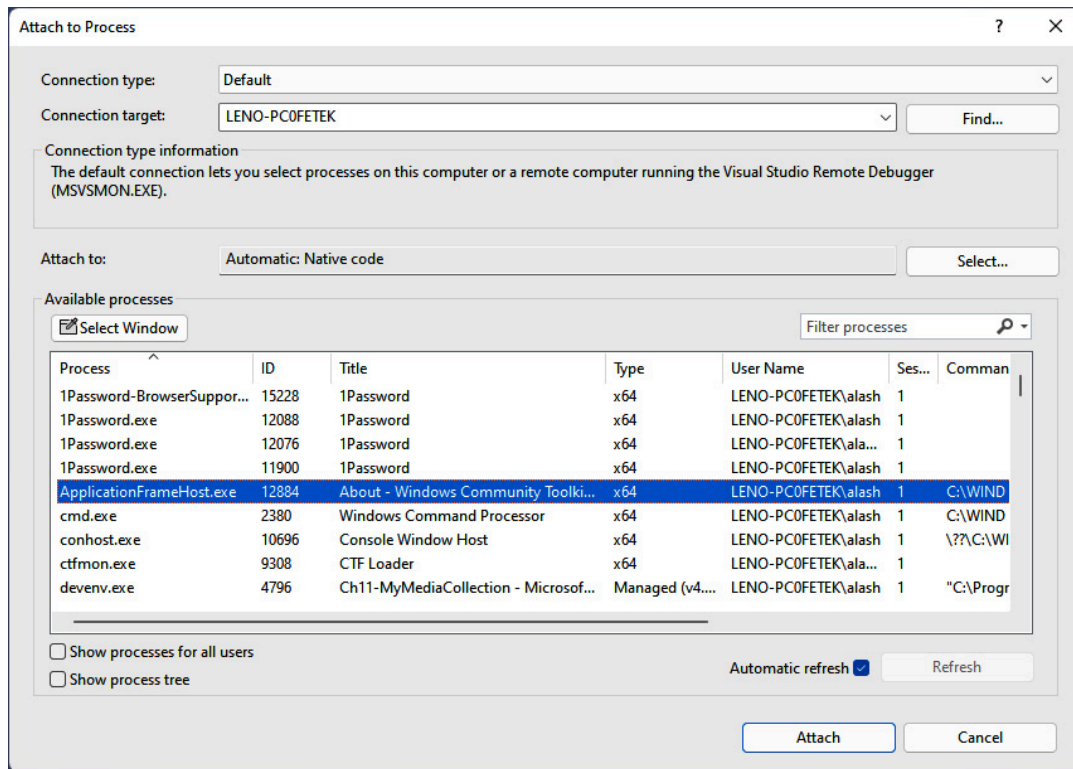


Figure 11.2 – Attaching to a running process to debug a packaged application

3. Click **Attach** and begin debugging as usual.

These are different ways to start a debugging session on a local application, but what if you want to debug your application running on another machine? Let's examine those options next.

Debugging remote applications

There are many reasons why you might want to debug an application on a remote machine. Sometimes you can only reproduce an error on one device. Perhaps there is an issue that only occurs on a specific device type or screen size. If you were working with a UWP project, there are some devices, such as Xbox, where you have to use remote debugging.

NOTE

*Before starting any remote debugging session, ensure that the target device has **developer mode** enabled. For more information about activating developer mode, you can read this Microsoft Docs article:*

<https://learn.microsoft.com/windows/apps/get-started/enable-your-device-for-development>.

To debug a remote installed application, you will use Visual Studio's **Debug Installed App Package** window again:

1. Open the window from **Debug | Other Debug Targets | Debug Installed App Package**.
2. Change **Connection Type** to **Remote Machine**:

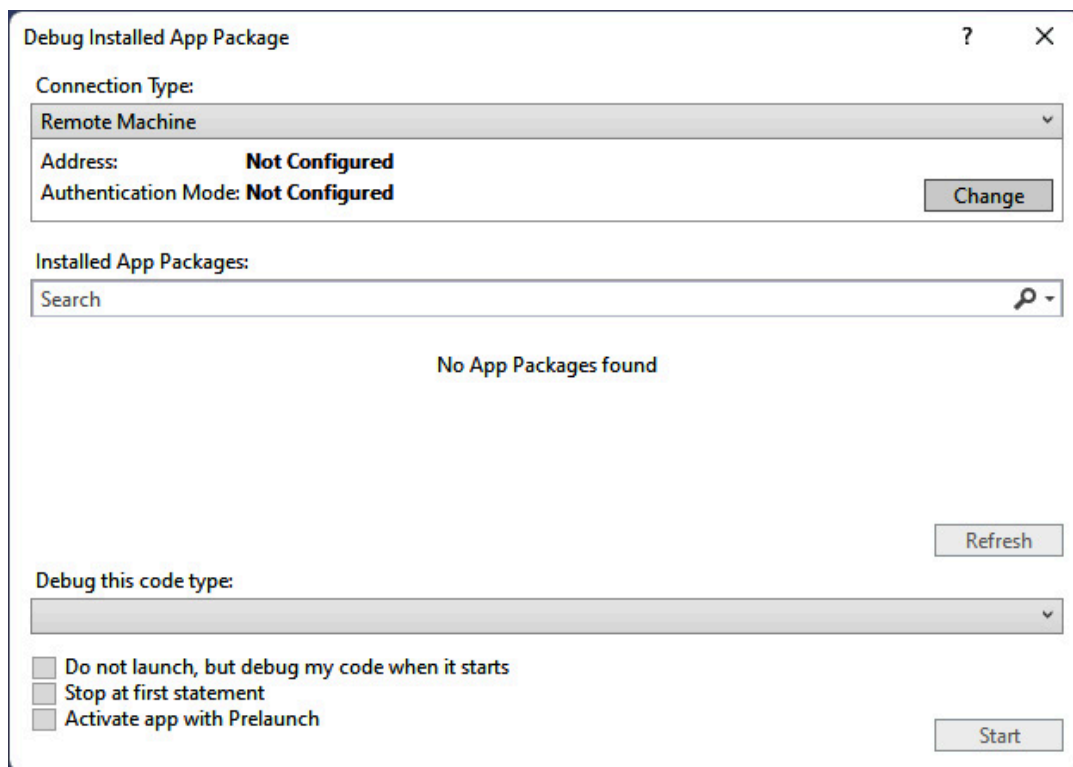


Figure 11.3 – Debugging an application package on a remote machine

3. Click the **Change** button in the **Connection Type** section to open the **Remote Connections** window.
4. Visual Studio will attempt to discover other Windows devices and list them in the **Auto Detected** section. If you see the device you want, select it to continue. If the device you want to debug isn't

shown, enter its IP address in the **Address** field in the **Manual Configuration** section and click **Select**:

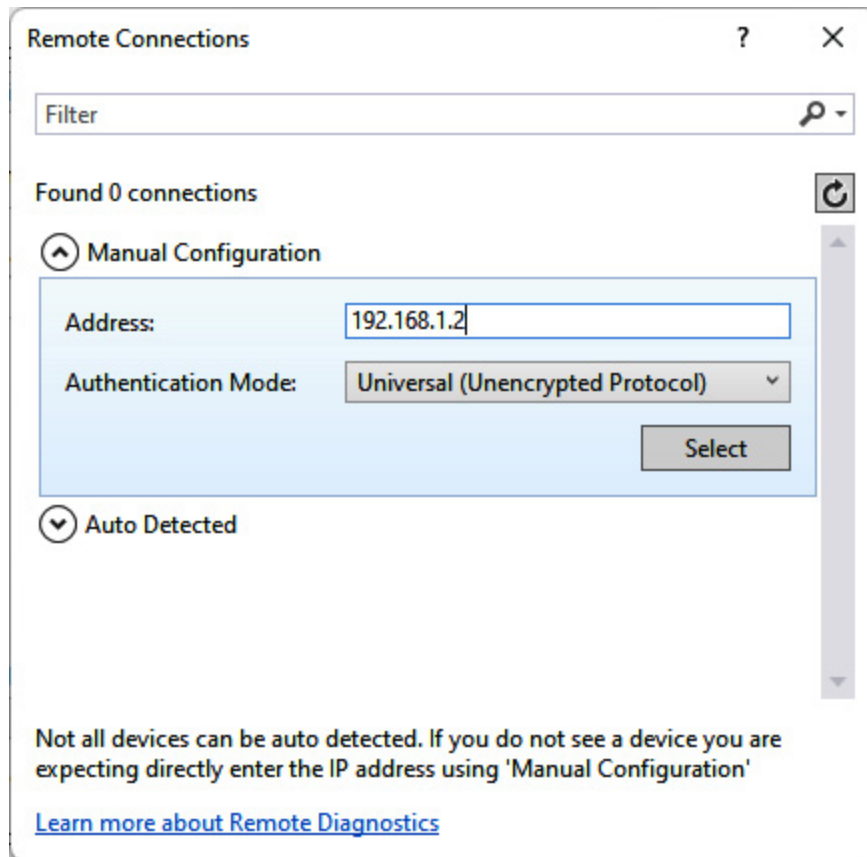


Figure 11.4 – Enter a manually configured remote connection for debugging

5. When you return to the previous window, you can select the application to debug from the list of installed app packages on the selected device.
6. Click **Start** to begin debugging, just as we did with the locally installed application in the previous section.

These techniques will allow you to connect to other Windows machines. For UWP applications, this includes other device types, such as Xbox, HoloLens, Surface Hub, and Windows IoT devices. Debugging options and resources can also be accessed from the **Debug** tab of the project's **Properties** page. Select **Open debug launch profiles UI** to open the **Launch Profiles** window:

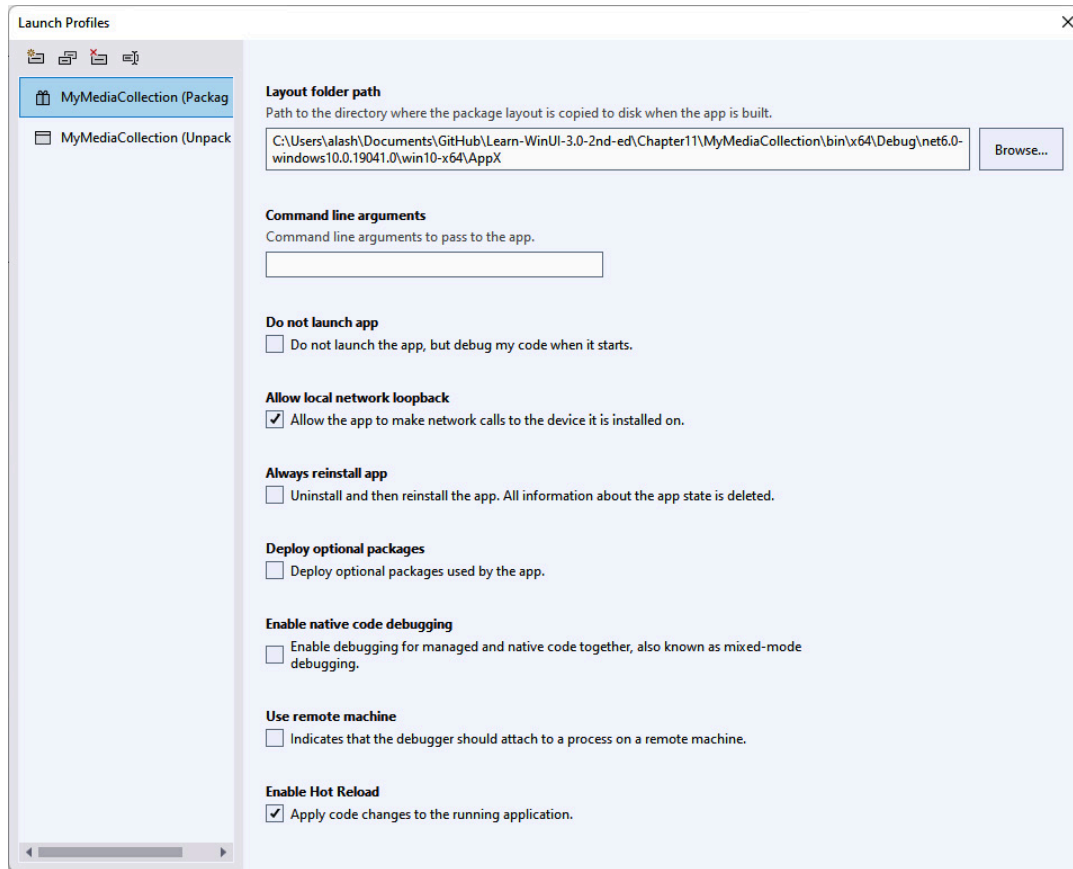


Figure 11.5 – The Launch Profiles window in Visual Studio

Let's shift gears and examine some common mistakes that can cause rendering issues in your application's UI.

Common XAML layout mistakes

There are many kinds of mistakes that XAML developers can make while coding the UI. Some of them won't be caught by the compiler. If you have a syntax error or an invalid **x:Bind** expression, these will fail while compiling, but many other issues will not.

The first source of common XAML layout mistakes we will explore is the **Grid** control.

Grid layout issues

Some mistakes related to the **Grid** control revolve around its rows and columns. Forgetting to set **Grid.Row** or **Grid.Column** on child controls

leads to overlapping or obscured elements at runtime. Similar issues can be seen when setting these values incorrectly or when working with **Grid.RowSpan** and **Grid.ColumnSpan**. One mistake that's not always immediately apparent is setting **Grid.Row** or **Grid.Column** to a value that's beyond the defined number of rows or columns.

Open **MainPage.xaml** in the **MyMediaCollection** project and remove the **Grid.Row** attribute from the **Border** control containing the **Button** controls at the bottom of the page. If you run the application, you will see the buttons move to the top of the page and overlap the controls in the header area.

Now restore the **Grid.Row** attribute but change the value to 5. If this were the last row in the grid, everything would look fine even though **Grid** only has four rows. Since 5 is greater than the number of available rows, the control is added to the last row in **Grid**. However, because we have **InfoBar** below the buttons in the fourth row, the buttons and the **InfoBar** overlap.

Problems when applying style

XAML **Style** resources are another common source of unintentional UI changes. When creating **Style** in **Resource**, you should be aware of how it will be applied to controls within the scope of that **Resource**.

Open **ItemDetailsPage.xaml** and review the **Page.Resources** section. This is where we created three **Style** elements for the current page. Each has a different **Target** type: **TextBlock**, **TextBox**, and **ComboBox**. However, they won't be applied to every control of those types on the **Page** because we also gave each **Style** an **x:Key**. That means that **Style** will only be applied to elements of that type when the **Style** property is explicitly set to that named resource. These are referred to as *explicit styles*:

```
<TextBlock Text="Name:" Style="{StaticResource  
AttributeTitleStyle}"/>
```

If you remove **x:Key** from **Style** in **Page.Resources**, that *implicit style* will apply to every control of the specified **Target** type on the **Page**, unless those controls have another explicit **Style** set. In a large application with styles declared at different scopes (**Application**, **Page**, or control), it can sometimes be difficult to determine which style has been applied to a control. We will see how to do this later in the chapter when discussing Visual Studio's **Live Property Explorer** window. It's a best practice to have implicit styles always inherit from an explicit style. This enables developers to inherit from a default style and reduces repeated implicit style attributes across elements.

Next, we will look at a third-party extension that can help find common XAML problems through **static code analysis**.

Improving your XAML with static code analysis

There is a free, open source extension for Visual Studio that, among other things, adds support for static code analysis to XAML files. **Rapid XAML Toolkit** (<https://rapidxaml.dev/>) provides XAML analyzers and code fixes for common issues and provides support for adding your own custom XAML analyzers.

NOTE

At the time of writing, Rapid XAML Toolkit for Visual Studio 2022 has not yet been released. It is being developed and should be available by the time you read this book. The current example illustrates how to use the extension currently available for Visual Studio 2019.

Let's install the tool in Visual Studio and see what kind of issues it can identify in a WinUI project:

1. In Visual Studio, go to **Extensions | Manage Extensions**.
2. The **Manage Extensions** window will appear. Ensure **Visual Studio Marketplace** is selected and search for **rapid xaml**:

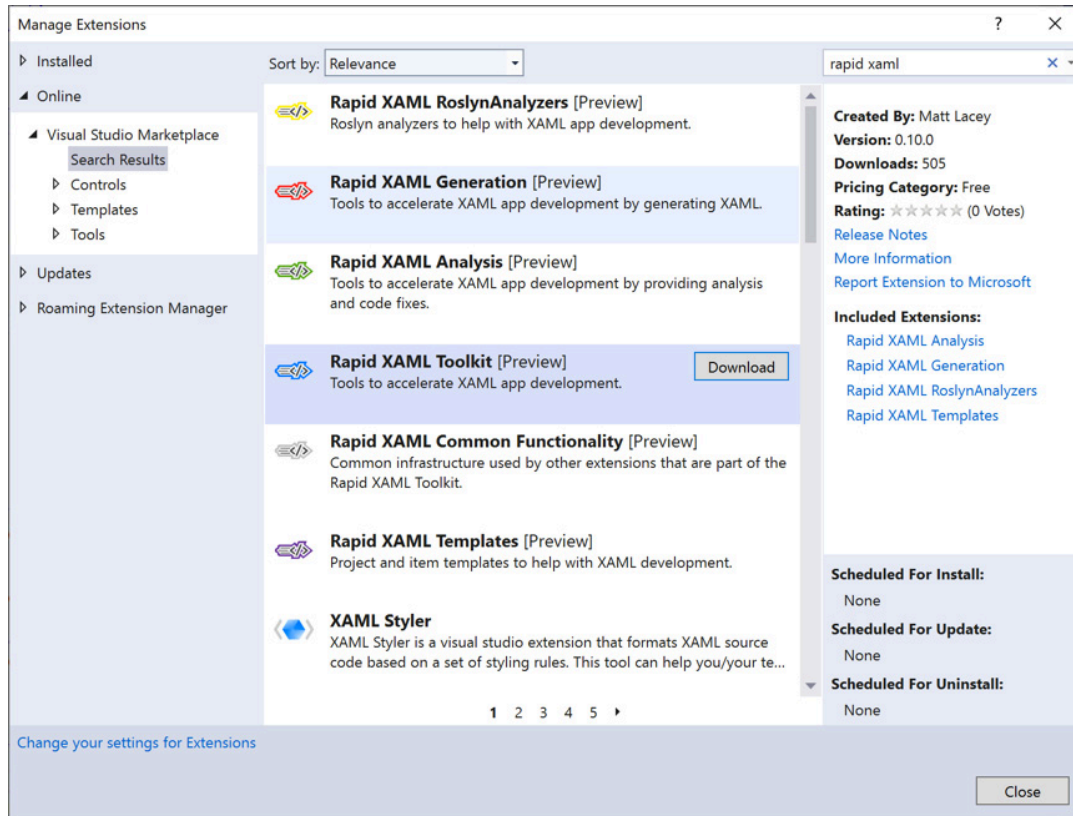


Figure 11.6 – Installing Rapid XAML Toolkit

3. Click **Download** to queue the extension for installation. When the download completes, you can close the **Manage Extensions** window and restart Visual Studio to complete the installation.
4. Now, when you view the **Error List** window in Visual Studio, there are a handful of warnings from Rapid XAML's analyzers:

Error List						
Entire Solution		0 Errors	8 Warnings	0 of 1 Message	Build + IntelliSense	Search Error List
Code	Description	Project	File	Line	Suppression State	
RXT200	TextBlock contains hard-coded Text value 'Item Details'.	WinUI2MediaCollection	ItemDetailsPage.xaml	35		
RXT200	TextBlock contains hard-coded Text value 'Name'.	WinUI2MediaCollection	ItemDetailsPage.xaml	50		
RXT200	TextBlock contains hard-coded Text value 'Media Type'.	WinUI2MediaCollection	ItemDetailsPage.xaml	53		
RXT200	TextBlock contains hard-coded Text value 'Medium'.	WinUI2MediaCollection	ItemDetailsPage.xaml	56		
RXT200	TextBlock contains hard-coded Text value 'Location'.	WinUI2MediaCollection	ItemDetailsPage.xaml	59		
RXT200	Button contains hard-coded Content value 'Save and Return'.	WinUI2MediaCollection	ItemDetailsPage.xaml	72		
RXT200	Button contains hard-coded Content value 'Save and Create New'.	WinUI2MediaCollection	ItemDetailsPage.xaml	76		
RXT200	Button contains hard-coded Content value 'Cancel'.	WinUI2MediaCollection	ItemDetailsPage.xaml	91		

Figure 11.7 – Viewing the new warnings from Rapid XAML Toolkit

5. Open **ItemDetailPage.xaml**, place your cursor over one of the green squiggles created by the code analyzer, and click the lightbulb icon:

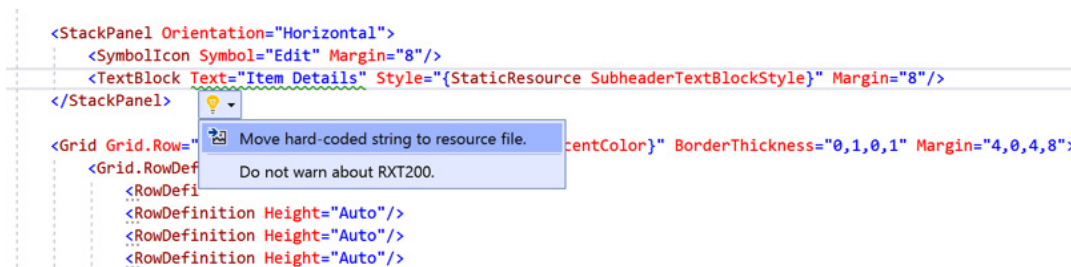


Figure 11.8 – View the quick fix for a code analyzer warning

Alternatively, you can right-click on the XAML and select **Rapid XAML | Move hard-coded string to resource file.** to fix all warnings of this type.

This is a list of other analyzers that are provided by the toolkit:

- **RXT101:** Use of a **Grid.Row** value without a corresponding **RowDefinition**
- **RXT102:** Use of a **Grid.Column** value without a corresponding **ColumnDefinition**
- **RXT103:** Use of a **Grid.RowSpan** value without a corresponding **ColumnDefinition**
- **RXT104:** Use of a **Grid.ColumnSpan** value without a corresponding **ColumnDefinition**
- **RXT150:** **TextBox** does not have **InputScope** specified
- **RXT160:** The **SelectedItem** binding should probably be **TwoWay**

- **RXT200:** The hardcoded string value that should be a resource
- **RXT300:** **Entry** does not have a **Keyboard** specified
- **RXT300:** Password entry does not have a **MaxLength** specified
- **RXT300:** **Image** lacks accessibility consideration
- **RXT300:** **ImageButton** lacks accessibility consideration
- **RXT401:** Handle both **Checked** and **Unchecked** events for a **CheckBox**
- **RXT402:** Use **MediaPlayerElement** in place of **MediaElement**
- **RXT451:** **x:Uid** should begin with an uppercase character
- **RXT452:** **Name** should begin with an uppercase character
- **RXT999:** Unknown error – something went wrong when parsing the XAML document

Rapid XAML Toolkit has many other features outside of analyzers and code fixes, and new features and analyzers are being added frequently. To see a list of upcoming features and fixes being considered, you can view the issues on GitHub:

<https://github.com/mrlacey/Rapid-XAML-Toolkit/issues>.

NOTE

Community-driven projects like this are always looking for contributors. It's a great way to get started with the open source community.

Another related topic that can be a common source of developer angst is debugging data binding. Let's see how we can avoid some common pitfalls in the next section.

Pinpointing data binding failures

While debugging data binding problems is not as difficult in WinUI and UWP as it is in WPF (if you use **x:Bind**-compiled bindings), there are still some *gotchas* to avoid. In this section, we will look at what can go wrong in views and ViewModels and how you can diagnose and fix the problems.

Common mistakes in data binding

If you use **x:Bind**, the compiler will evaluate whether you're binding to a valid source and can give you the peace of mind of knowing that your views and ViewModels are hooked up correctly, but there is still a lot that can go wrong. Let's review a few of the most common mistakes.

Selecting the best binding mode

We have seen in previous chapters that the default mode for most controls with **x:Bind** is **OneTime**, while the default for **Binding** is **OneWay**. Defaulting to **OneTime** helps with performance as many read-only properties are only ever set when the view is first created. However, if you forget to change this for controls bound to data that changes as the user is interacting with the page, you may not immediately realize why the data in the UI isn't updating.

When you're binding controls that need data to flow in both directions, remember to set **Mode** to **TwoWay**. We used this in the **MyMediaCollection** project with the **ComboBox.SelectedItem** property to filter the collection by media type.

Triggering PropertyChanged notifications

By binding to a ViewModel that properly implements **INotifyPropertyChanged** for all its public properties, issues related to **PropertyChanged** are not common. Using MVVM Toolkit makes it even simpler. Problems can still arise if ViewModel code outside of one of these properties updates the property's value by setting the **private** backing variable. This will update the property's value without notifying the view. If you are using MVVM Toolkit, it will call out this issue when it generates its code in the hidden partial class. You can avoid this by always using the **public** property to update the value. If there is a good reason for not updating the property directly, then a

PropertyChanged event should be manually triggered for that property after updating the value. It is a best practice to use the **nameof** method in C# to ensure you use a property name that exists. Misspelled property names will be caught at compile time and highlighted in the editor. You can also use **CallerMemberNameAttribute** in .NET:

<https://learn.microsoft.com/dotnet/api/system.runtime.compilerservices.callermembernameattribute>. An exception will be raised if your application tries to raise a property change notification for a property that does not exist.

Working with **ObservableCollection<T>**

ObservableCollection<T> serves an important role. Lists in the view will stay synchronized with the collections if they are used correctly. There are a few practices to avoid when using observable collections.

Do not replace the entire value of an observable collection. The element on the view bound to the property will still be bound to the original collection. Any subsequent changes to the collection will not be reflected in the view. While you can work around this by triggering a **PropertyChanged** notification, this can have performance implications with larger collections. It can also be jarring to the user, as most controls will reset the current view to the beginning of the list. Set ViewModel properties that use **ObservableCollection<T>** to be read-only to avoid accidentally resetting the entire collection. The one exception to this is if you know the collection will be completely repopulated. Removing and re-adding a large number of items individually in a list can result in a poor user experience.

Do not use LINQ to modify observable collections. LINQ expressions do not operate by calling the **Add** and **Remove** methods on the observable collections. They do not even return **ObservableCollection<T>**. If you use LINQ and convert the results back to

ObservableCollection<T>, you will be back to replace the entire collection, which was just discussed.

Here is an example using LINQ. This will cause the view to stop receiving **CollectionChanged** notifications because LINQ does not return an **ObservableCollection**:

```
_entryList = from entry in _entryList
               where entry.Lastname.StartsWith("J")
               select entry;
```

This example does the same thing. It removes any entries with a **Lastname** starting with J. By using the **Remove** method and not changing the entire collection, the **CollectionChanged** events are preserved:

```
for (int i = _entryList.Count - 1; i >= 0; i--)
{
    Entry entry = _entryList[i];
    if (!entry.Lastname.StartsWith("J"))
    {
        _entryList.Remove(entry);
    }
}
```

Finally, it's best not to use observable collections for lists that do not change. The extra overhead they bring is not needed for static data. Bind to an **IList<T>**, **IEnumerable<T>**, or better yet, a traditional array of items instead. Check out this *C# 101* video to learn when to leverage arrays, lists, and collections in .NET:

<https://learn.microsoft.com/shows/csharp-101/csharp-arrays-list-and-collections>.

Using the XAML Binding Failures window

Visual Studio 2019 version 16.7 introduced the new **XAML Binding Failures** window and a **Binding failures** indicator on the in-app toolbar. These are for failures in **Binding**, not **x:Bind**, as those issues are checked and caught by the compiler. Clicking the red indicator will take you to the **XAML Binding Failures** window while debugging. Binding failures were always available in the **Output** window while debugging, but they could be difficult to find. The new, dedicated window provides sorting and searching. Similar failures are also grouped to make it easy to address related items together:



Figure 11.9 – Checking the in-app toolbar for binding failures

Let's give it a try. Open the **XamlDebugging** solution from the GitHub repository for this chapter and follow along with these steps:

1. Open **MainWindow.xaml** and review the XAML. **Window** contains a **StackPanel1** with two **TextBox** child elements:

```
<StackPanel DataContext="Test" Background="LightGray">
  <TextBox x:Name="Text1"
    Text="{Binding Path=SomeText,
      Mode=TwoWay}"/>
  <TextBox Text="{Binding ElementName=Text1,
    Path=Text, Mode=OneWay}"/>
</StackPanel>
```

DataContext does not exist, so what do you think will be reported in the binding failures? Let's find out!

2. Run the application and look at the in-app toolbar. There is an indication that we have one binding failure.
3. Click the red indicator on the toolbar to open the **XAML Binding Failures** window in Visual Studio:

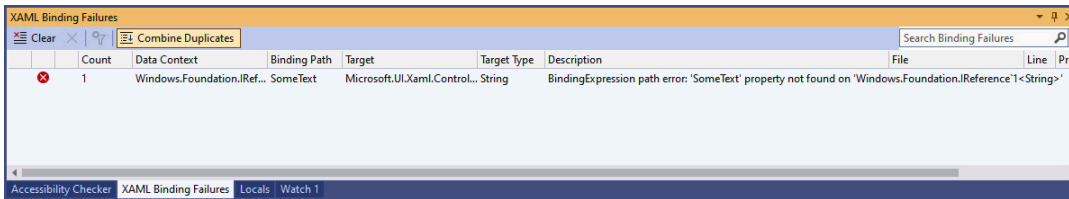


Figure 11.10 – The XAML Binding Failures window

The **Description** text in the window tells us **BindingExpression path error: ‘SomeText’ property not found on ‘Windows.Foundation.IReference`1<String>’**. This is useful information. It would be more useful if there was something telling us directly that the data context itself is not valid. However, if you had a dozen controls using this data context, and all the bindings were failing, it would become clear that there is a problem with the data context and not the individual bindings.

4. Let’s make sure the other binding is working correctly. Enter some text into the first **TextBox**:

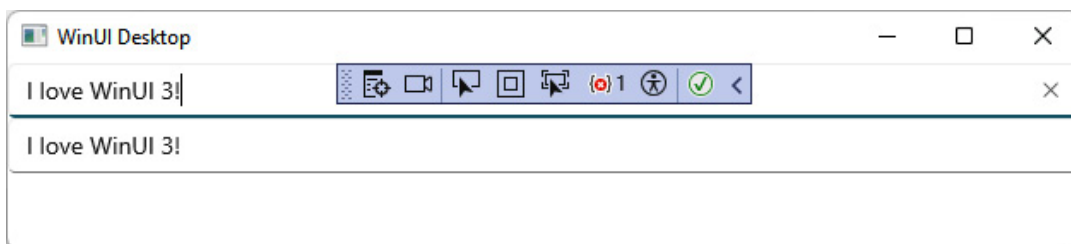


Figure 11.11 – Binding TextBoxes with ElementName

The text you enter should be duplicated in the second **TextBox** as your type. Our binding to **ElementName** is working as expected.

I will leave fixing the data source as an exercise for you. Add a valid data source to the code behind it and see whether the binding error is cleared up.

Now, let’s explore a few other XAML debugging tools available from the in-app toolbar.

Debugging live data with Live Visual Tree and Live Property Explorer

While the **XAML Binding Failures** window is new to Visual Studio, the in-app toolbar has been available to XAML developers since 2015. The toolbar floats over the active window in your application while you are debugging. There are several parts to the toolbar:

- **Go to Live Visual Tree:** Opens the **Live Visual Tree** window.
- **Show in XAML Live Preview:** This opens the **XAML Live Preview** window in Visual Studio.
- **Select Element:** Allows you to select an element in **Live Visual Tree** by clicking on it in the active window.
- **Display Layout Adorners:** This will highlight the element in the UI that is currently selected in **Live Visual Tree**.
- **Track Focused Element:** While the **Live Visual Tree** window is open, toggling this on will indicate in **Live Visual Tree** which element currently has focus on the UI.
- **Binding failures:** Indicates the number of current binding failures and opens the **XAML Binding Failures** window when clicked.
- **Scan for Accessibility Issues:** This opens the **Accessibility Checker** window and runs an accessibility check on the UI elements currently visible.
- **Hot Reload:** Indicates whether **XAML Hot Reload** is currently available. Clicking it will open the documentation on Microsoft Learn.

Let's explore a few of the most commonly used debugging tools. If you wish to get more information on the remaining tools, you can visit this page about XAML tools on Microsoft Learn:

<https://learn.microsoft.com/visualstudio/xaml-tools/>. Or, view this *Visual Studio Toolbox* episode about using the XAML tools in Visual

Studio: <https://learn.microsoft.com/shows/visual-studio-toolbox/new-xaml-features-in-visual-studio>. We'll start with XAML Hot Reload.

Coding with XAML Hot Reload

XAML Hot Reload is a simple but powerful feature. Before Visual Studio 16.2, it was known as **XAML C# Edit & Continue**. Hot reload has been available to web developers for a while longer, and the name change for XAML helps to clear up some confusion for developers familiar with the concept in web development. The idea of hot reload is that you can make changes to your UI and the changes are reflected in the running application without having to stop debugging and recompile. Let's try it with the **XamlDebugging** solution:

1. Start by running the application again. Make sure that the **Hot Reload** indicator in the toolbar shows that it is enabled. It should only be disabled if you are running an unsupported project type.
2. Now let's make a change to **MainWindow.xaml**. We're going to change the **Background** color of **StackPanel** to **LightGray**:

```
<StackPanel DataContext="Test"
    Background="LightGray">
    <TextBox x:Name="Text1"
        Text="{Binding Path=SomeText,
            Mode=TwoWay}"/>
    <TextBox Text="{Binding ElementName=Text1,
        Path=Text, Mode=OneWay}"/>
</StackPanel>
```

3. Save the file and look at the running application:

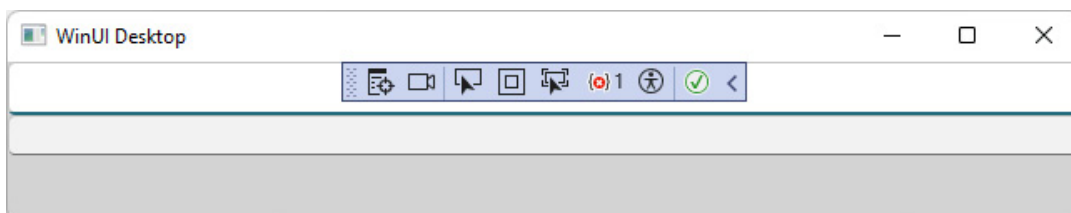


Figure 11.12 – Changing the UI without restarting the application

The background of the window is now light gray.

This is a huge time-saver when building a new UI. For a list of known limitations, see this Microsoft Learn article:

<https://learn.microsoft.com/troubleshoot/developer/visualstudio/tools-utilities/xaml-hot-reload-troubleshooting#known-limitations>.

Now let's look at another powerful debugging tool, the **Live Visual Tree** window.

Debugging with Live Visual Tree and Live Property Explorer

The **Live Visual Tree** window allows developers to explore the elements in the current window of an application's XAML **visual tree**. It is available to UWP and WinUI applications and other XAML frameworks. Let's step through using **Live Visual Tree** and related XAML debugging tools:

1. While debugging the **XamlDebugging** project, open the **Live Visual Tree** window. The visual tree contains the hierarchy of controls in the window with the **Show Just My XAML** button selected by default in the **Live Visual Tree** window's toolbar:

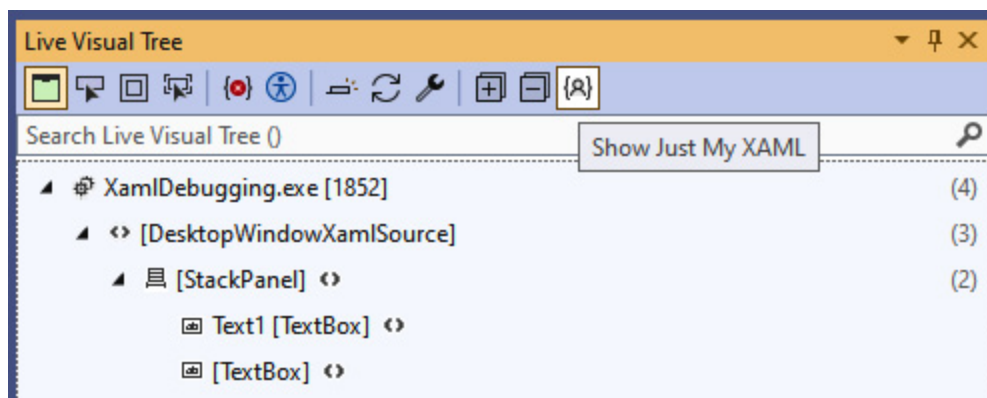


Figure 11.13 – Viewing Live Visual Tree for the XamlDebugging project (Show Just My XAML selected)

2. If you want to view the entire visual tree for your window, de-select **Show Just My XAML** on the toolbar and the tree will refresh:

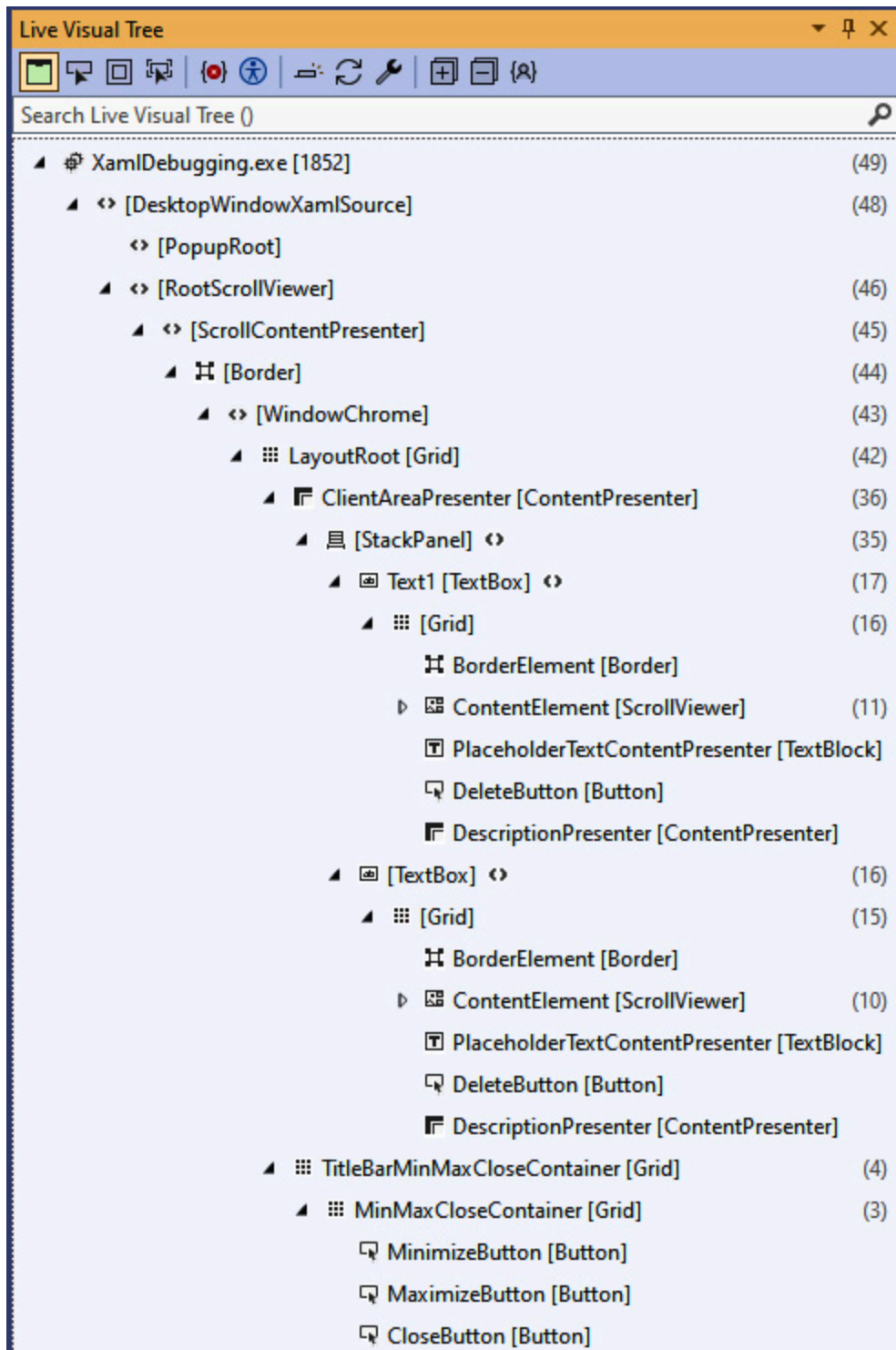


Figure 11.14 – Viewing Live Visual Tree for the XamlDebugging project (all XAML)

There's a lot more going on here. With **Show Just My XAML** de-selected, you can see the elements that make up each control. These are

the contents of the **control templates**. A **TextBox** is made up of 17 child elements with a **Grid** at the root of its template. This is a great way to learn how the controls we use are composed. Working with and modifying these templates is beyond the scope of this chapter, but I encourage you to explore them on your own. ***Chapter 7**, **Fluent Design System for Windows Applications***, has additional information on using default styles and theme resources. This Microsoft Learn article on control templates is another great place to start:

<https://learn.microsoft.com/windows/apps/design/style/xaml-control-templates>. For now, let's switch back to the **Show Just My XAML** view.

3. From **Live Visual Tree**, right-click on a node to navigate to the XAML markup for the selected item by selecting **View Source**.
4. Next, right-click a node and select **Show Properties** to show the **Live Property Explorer** window, where you can inspect the current properties of the selected element. You can see how the properties are grouped based on where they have been set. Here, some are set by **Local** changes and others are set based on the **Style** for a **TextBox**. If you had multiple levels of inherited explicit styles and an implicit style, those would all be grouped here:

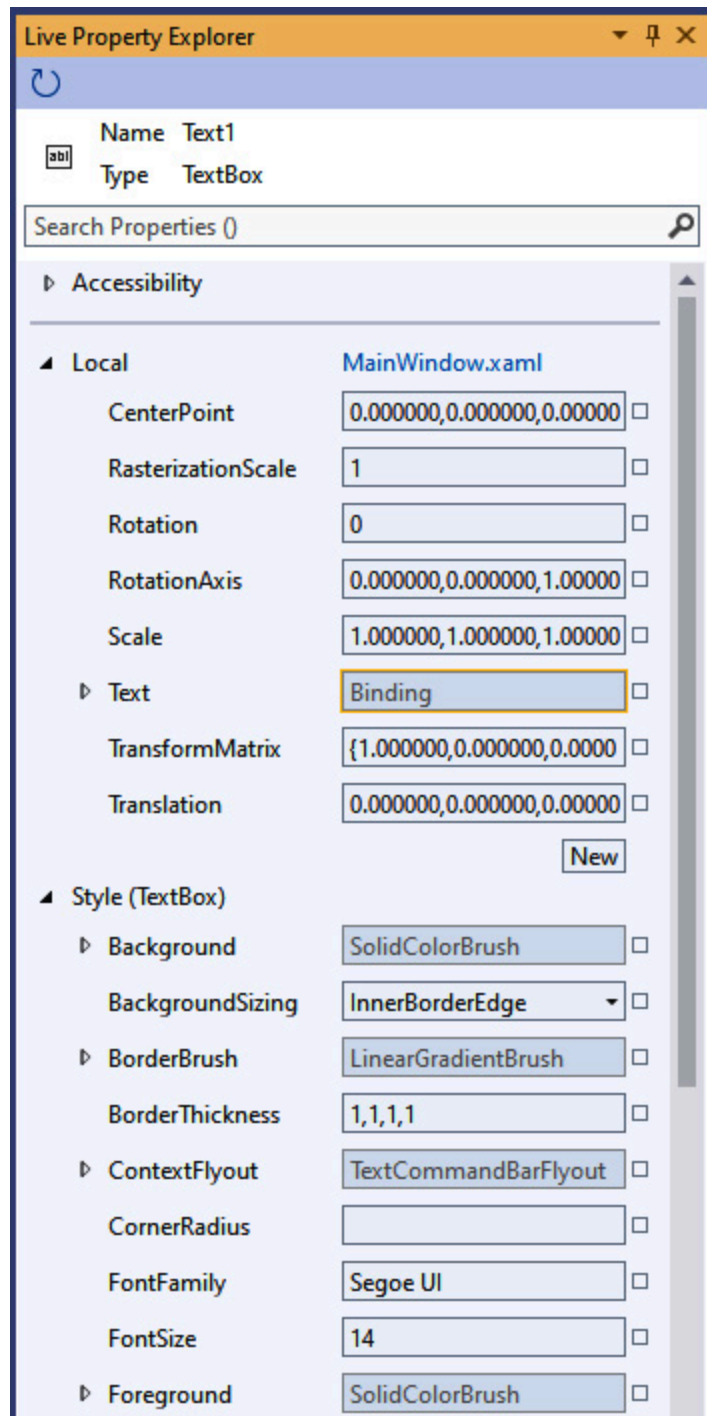


Figure 11.15 – Viewing the Live Property Explorer window for the Text1 TextBox

5. Look at the **Text** property of **Text1**. The value is **Binding**, meaning that the value is set to a **Binding** markup extension. Expand the **Text** property node to view the details of the binding:

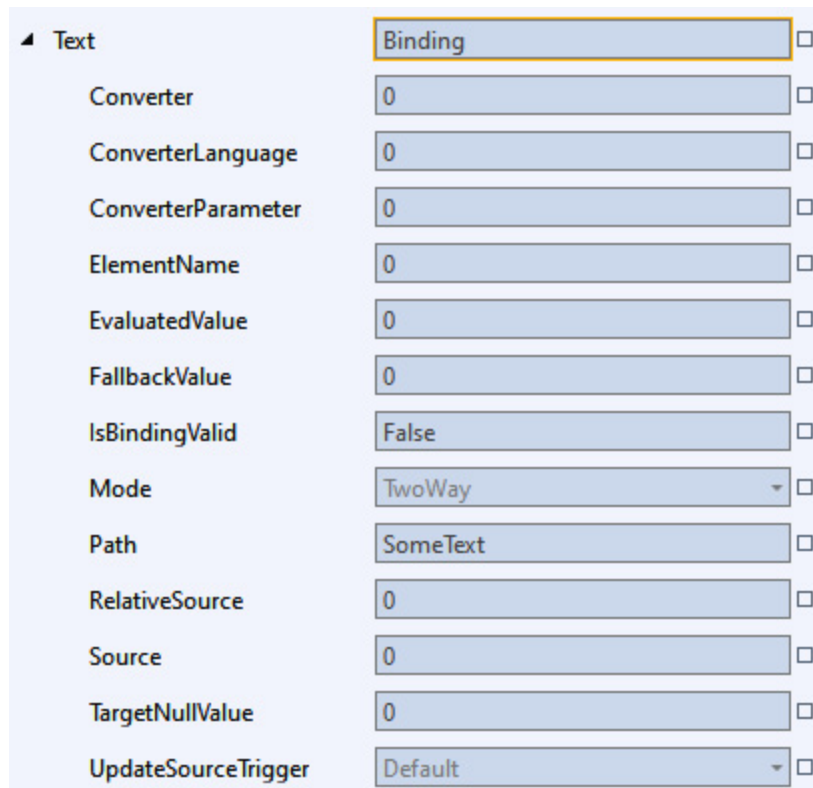


Figure 11.16 – View the binding details for the Text property of Text1

There is some very helpful information here. We can see the **Path** and **Mode** that were set in the XAML, the ones we did not set, and yet others that are read-only. **IsBindingValid** is **False** in this project, but it may be **True** if you fixed your data context from the previous section. When the binding is valid, **EvaluatedValue** will contain the current value for the **Text** property.

6. Select the second **TextBox** element in the tree and view the **Live Property Explorer** window. You can see that the default value for **Background** is **SolidColorBrush**.
7. Expand the **Background** node to view the resource details:

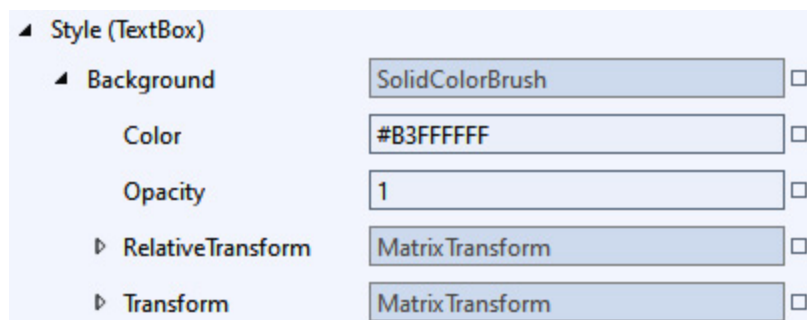


Figure 11.17 – Viewing the property details of Page.Background

8. Try changing the **Color** property to white and see what happens in the XAML code and to the running application. The UI in the app should update, but the XAML file will remain unchanged.
9. Next, try adding markup to **MainWindow** to set the **MaxWidth** property of the second **TextBox** to **100**. What happens in **Live Property Explorer**? Can you find this property and value? Try using the search field. You can remove the **MaxWidth** when you're finished.
10. Returning to **Live Visual Tree**, you can view the depth of the visual tree to the right of each node's name. Remember that this number is not the true depth of the visual tree when **Show Just My XAML** is enabled. It is the depth of the visible nodes. This is important to note because the depth and complexity of the visual tree impact the performance of the UI rendering. **Grid** is more complex than **StackPanel**. **StackPanel** is preferred for layout if you are simply laying out a few elements horizontally or vertically. This unidirectional layout is what **StackPanel** was built to handle. As a rule of thumb, you should use the right control for the job and always keep performance in mind.
11. Now select the active node from the tree in your XAML file and enable **Select Element in the Running Application** from the toolbar. The **Track Focused Element** button on the toolbar will highlight the current tree node in the application's window at runtime. This option is also available from the in-app toolbar.
12. To toggle the in-app toolbar on/off, you can use the **Enable in-app Toolbar** button on the **Live Visual Tree** toolbar.

I encourage you to spend some time in these windows the next time you are debugging your application. Use them to help find issues with binding, resources, or custom control templates.

Now let's wrap up the chapter and review what we've learned.

Summary

In this chapter, we have covered some essential tools and techniques for debugging your XAML applications. We learned how to debug installed applications on a local or remote PC. If you are developing WPF or UWP applications, tools such as **Live Visual Tree**, **Live Property Explorer**, and **Rapid XAML Toolkit** will all work for those projects as well. Leveraging these tools will shorten the time you spend debugging and help you deliver higher-quality software.

In the next chapter, we will explore the **WebView2** browser control. You will learn how to use **WebView2** to embed an ASP.NET Core Blazor **single-page application (SPA)** inside a WinUI 3 desktop application.

Questions

1. How can you debug a UWP application that is running on an Xbox?
2. How can you debug an application that is currently running on your machine?
3. How can you launch an application package installed on a remote machine for debugging?
4. What feature in Visual Studio allows you to change XAML properties at runtime and see them reflected in the running application?
5. Which window will show a hierarchy of the elements in the current window?
6. What is the default binding mode for most control properties with **x:Bind**?
7. Where can you view the runtime properties for the control currently selected in **Live Visual Tree**?